

Exercise Sheet 10

Efficient Computing/Parallel Computing 2 in SS 2026

Date of issue: 30.6.2026

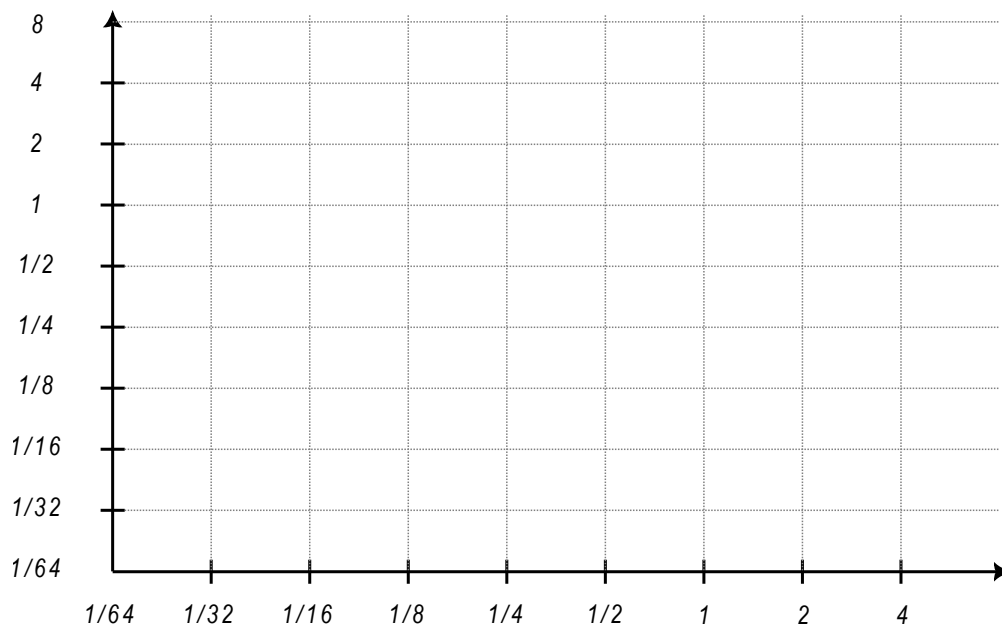
Deadline: 6.7.2026

The exercises marked with ★ are relevant for admission to the examination and must be submitted in electronic form in Moodle by the submission deadline.

Exercise 10.1 (Comparison of two algorithms using the roofline model).

Design a roofline model for a machine with a peak compute performance of $\pi = 3 \frac{\text{GFLOP}}{\text{s}}$. Assume that the data transfer between the compute device and the main memory is limited by a peak memory bandwidth of $\beta = 8 \frac{\text{GB}}{\text{s}}$.

- a) Draw the roofline model in the following diagram, specify the axes and their units. Also indicate how π and β are reflected in this diagram. Mark the regions in which algorithms are limited by their peak compute performance (compute-bound) or by their peak memory bandwidth (memory-bound).



- b) Determine the operational intensity for the following two algorithms. Identify these two intensities in the previous diagram and indicate for both algorithms in which region (compute-bound, memory-bound) they fall.

File: `Example1.c` 

```
float x[N], y[N];
for (int i=0; i<N; i++){
```

```
    x[i] = x[i] + y[i];  
}
```

File: Example2.c 

```
float t = 0.0, x[N];  
for (int i=0; i<N; i++){  
    t = t + x[i]*x[i];  
}
```

Exercise 10.2 (*CUDA matrix addition with a two-dimensional grid*).

Write a CUDA program that computes the matrix addition

$$C \leftarrow A + B$$

for two matrices $A, B \in \mathbb{R}^{m \times n}$, where $m = 1000 \cdot i$ and $n = 700$.

- Write a CUDA kernel `matrix_add`. Each thread shall compute one matrix entry
`C[row * cols + col] = A[row * cols + col] + B[row * cols + col]`.
Assume row-major storage.
- Use the two-dimensional thread and block indices to compute the row and column handled by each thread.
- Choose a two-dimensional block and grid configuration, for example 16×16 threads per block. Make sure that the kernel works even if the grid contains threads outside the matrix dimensions.
- Write the necessary code: allocate memory on the host and on the device, initialize A and B , copy data to the device, launch the kernel, and copy C back to the host.
- Validate your results and make run-time experiments for varying block-sizes.